

Sentence Memorability Graphs

```
raw_dataset <- read_csv(
  "sentence_memorability_data.csv",
  na = c("", "NA", "N/A"),
  show_col_types = FALSE
)

aggregated_matrix <- raw_dataset %>%
  filter(isTarget == TRUE) %>%
  arrange(participant_ID, Stimulus, Timestamp) %>%
  mutate(
    # mapping conditions
    Word_Condition = case_when(
      str_detect(Stimulus, "^HH_") ~ "HH",
      str_detect(Stimulus, "^HVL_") ~ "HL",
      str_detect(Stimulus, "^LVH_") ~ "LH",
      str_detect(Stimulus, "^LVL_") ~ "LL"
    ),
    Word_Condition = factor(Word_Condition, levels = c("HH", "HL", "LH", "LL")),

    Sentence_Voice = case_when(
      str_detect(Stimulus, "_A$") ~ "Active",
      str_detect(Stimulus, "_P$") ~ "Passive",
      TRUE ~ NA_character_
    ),
    Sentence_Voice = factor(Sentence_Voice, levels = c("Active", "Passive")),

    Accuracy_IR = suppressWarnings(as.numeric(Accuracy_IR)),
    Accuracy_WR = suppressWarnings(as.numeric(Accuracy_WR)),

    # store previous IR result
    prev_IR = lag(Accuracy_IR)
  ) %>%
  group_by(Stimulus, Word_Condition, Sentence_Voice) %>%
  summarise(
    Participants = n_distinct(participant_ID),
    # IR components
    Total_Repeats = sum(isRepeat == TRUE & Event == "Sentence shown", na.rm = TRUE),
    Total_Denom = sum(is.na(isRepeat) & Event == "Sentence shown"),
    Hits = sum(Event == "IR pressed" & isRepeat == TRUE & Accuracy_IR == 1, na.rm = TRUE),
    False_Alarms = sum(Event == "IR pressed" & is.na(isRepeat), na.rm = TRUE),

    # IR metrics
    Hit_Rate = if_else(Total_Repeats > 0, Hits / Total_Repeats, 0),
    False_Alarm_Rate = if_else(Total_Denom > 0, False_Alarms / Total_Denom, 0),
    IR_Memorability = Hit_Rate - False_Alarm_Rate,
```

```

# WR components (after correct IR)
WR_correct = sum(Event == "WR pressed" & Accuracy.WR == 1 & prev_IR == 1, na.rm = TRUE),
WR_total   = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),

# WR metric
WR_Accuracy = if_else(WR_total > 0, WR_correct / WR_total, 0),

.groups = "drop"
) %>%
mutate(WR_Accuracy = replace_na(WR_Accuracy, 0))

aggregated_matrix <- aggregated_matrix %>%
  filter(Participants >= 15)

```

```

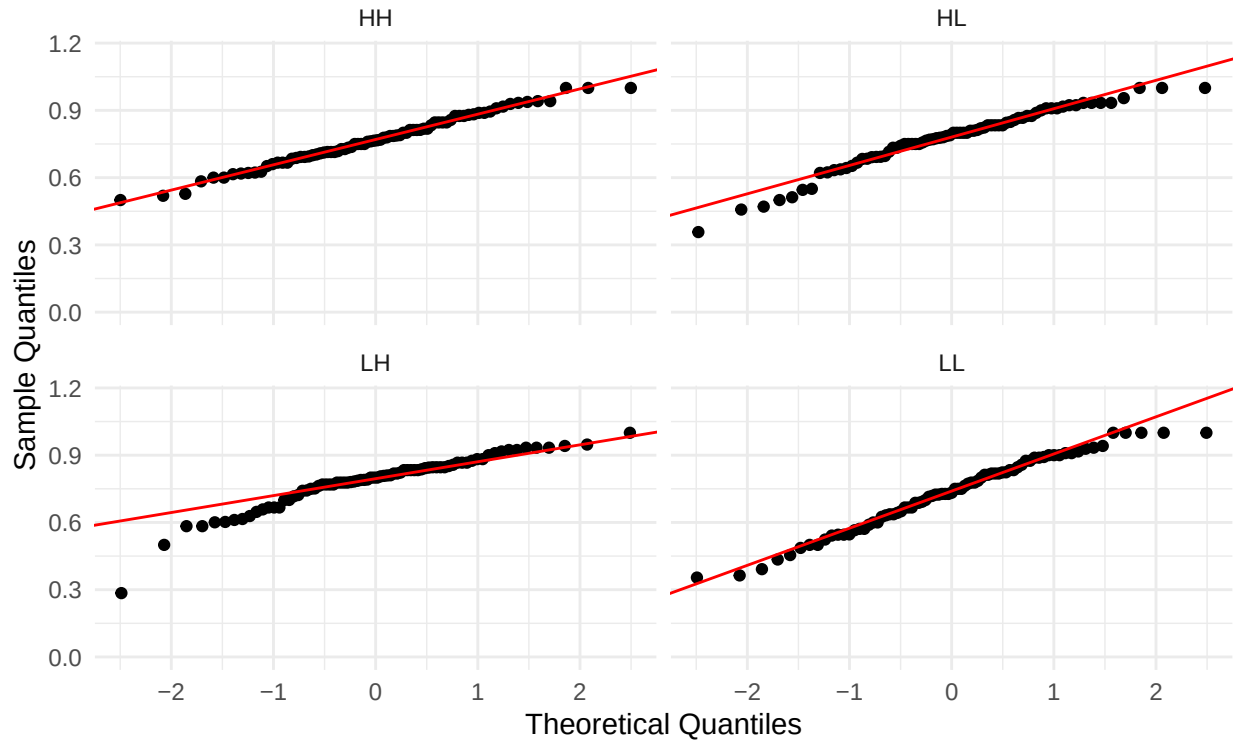
# Testing for Normality
# QQ Plots
library(ggplot2)

ggplot(aggregated_matrix, aes(sample = IR_Memorability)) +
  stat_qq() +
  stat_qq_line(color = "red") +
  facet_wrap(~ Word_Condition, ncol = 2) + # This creates the 4-plot grid
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "Q-Q Plots: IR Memorability by Word Condition",
       subtitle = "Checking for Normality across HH, HL, LH, and LL",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()

```

Q-Q Plots: IR Memorability by Word Condition

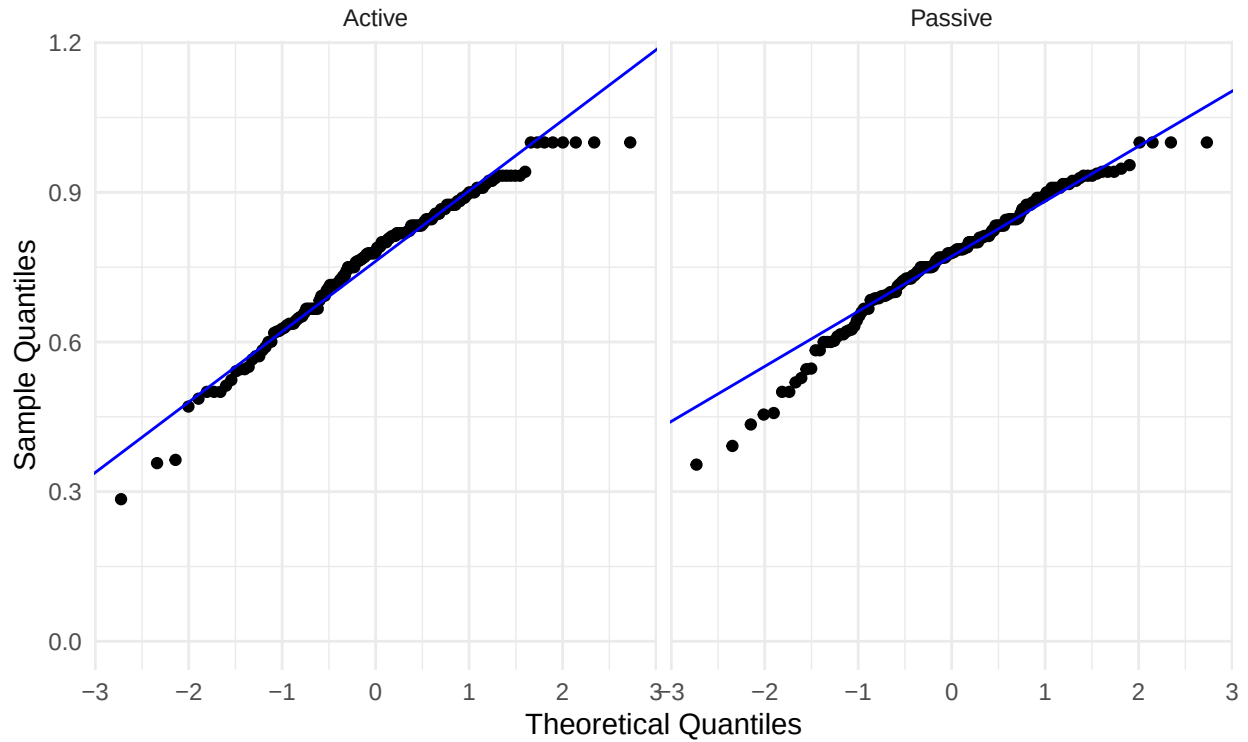
Checking for Normality across HH, HL, LH, and LL



```
ggplot(aggregated_matrix, aes(sample = IR_Memorability)) +
  stat_qq() +
  stat_qq_line(color = "blue") + # Using blue to distinguish from the Word Condition plots
  facet_wrap(~ Sentence_Voice, ncol = 2) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "Q-Q Plots: IR Memorability by Sentence Voice",
       subtitle = "Checking for Normality across Active and Passive Voices",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()
```

Q-Q Plots: IR Memorability by Sentence Voice

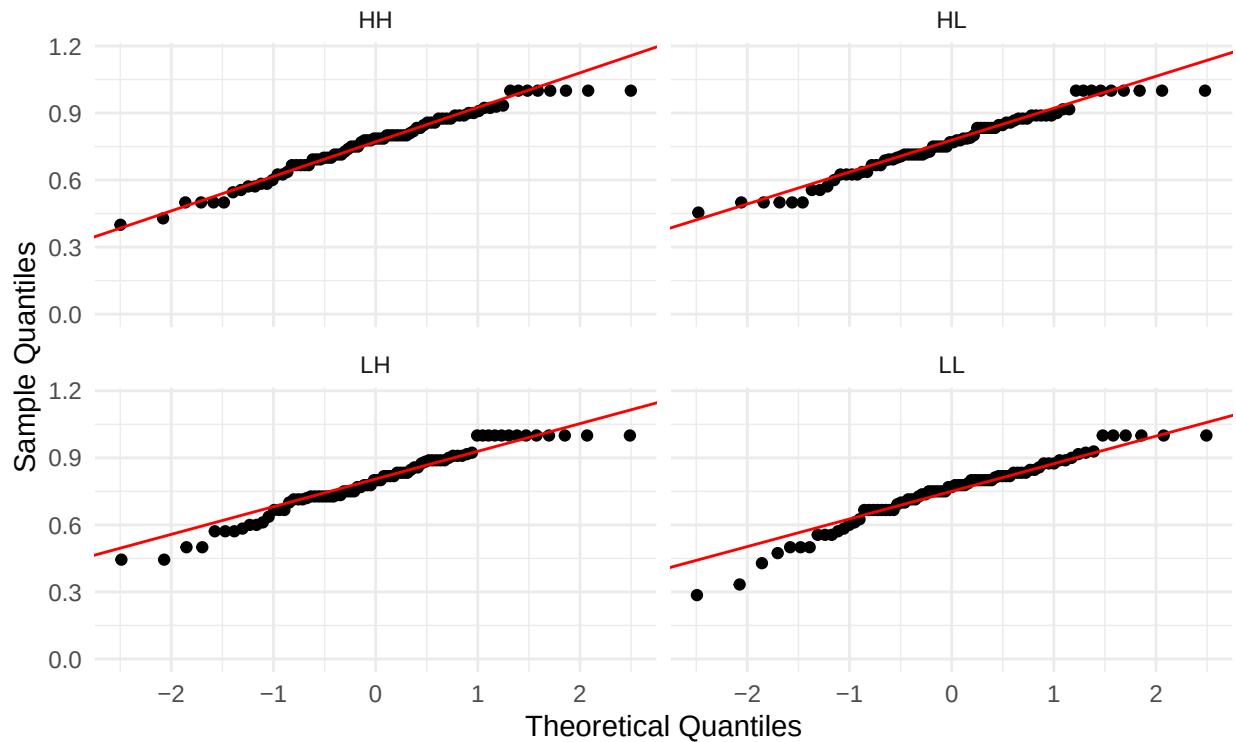
Checking for Normality across Active and Passive Voices



```
ggplot(aggregated_matrix, aes(sample = WR_Accuracy)) +
  stat_qq() +
  stat_qq_line(color = "red") +
  facet_wrap(~ Word_Condition, ncol = 2) + # This creates the 4-plot grid
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "Q-Q Plots: WR_Accuracy by Word Condition",
       subtitle = "Checking for Normality across HH, HL, LH, and LL",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()
```

Q-Q Plots: WR_Accuracy by Word Condition

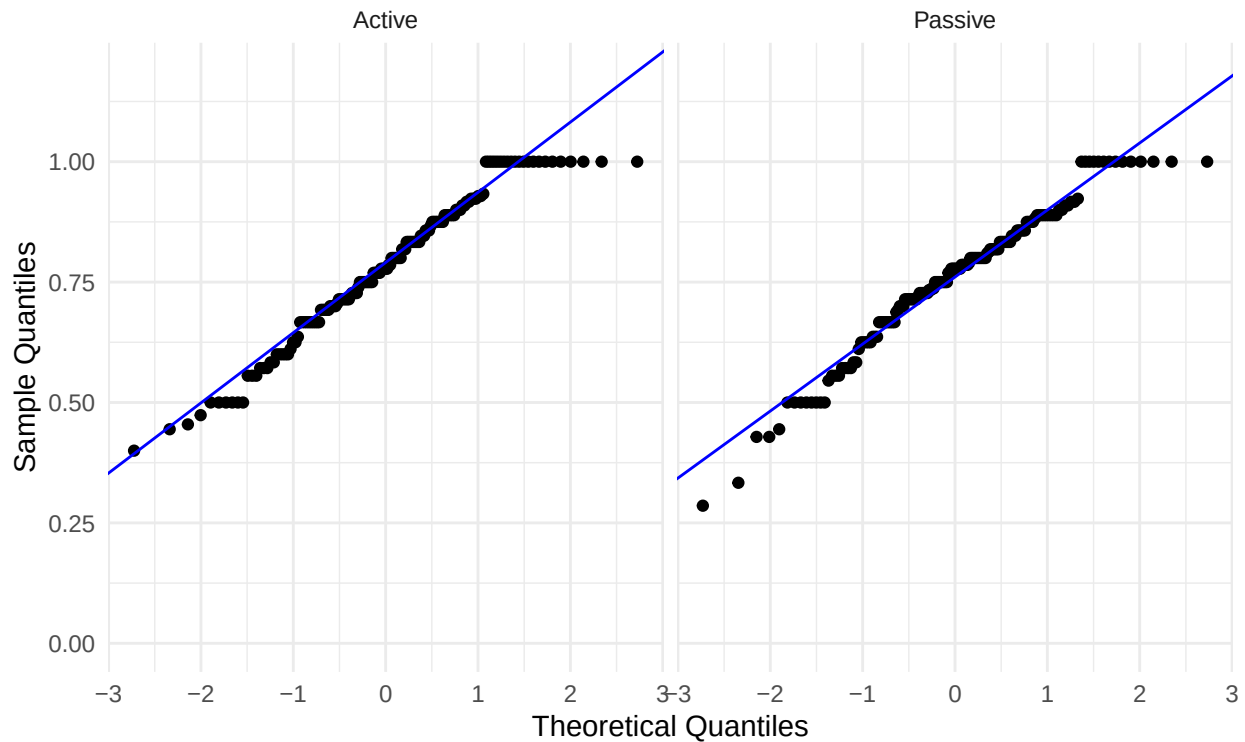
Checking for Normality across HH, HL, LH, and LL



```
ggplot(aggregated_matrix, aes(sample = WR_Accuracy)) +
  stat_qq() +
  stat_qq_line(color = "blue") + # Using blue to distinguish from the Word Condition plots
  facet_wrap(~ Sentence_Voice, ncol = 2) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "Q-Q Plots: WR_Accuracy by Sentence Voice",
       subtitle = "Checking for Normality across Active and Passive Voices",
       x = "Theoretical Quantiles",
       y = "Sample Quantiles") +
  theme_minimal()
```

Q-Q Plots: WR_Accuracy by Sentence Voice

Checking for Normality across Active and Passive Voices



```
# Anderson Darling
library(nortest)

aggregated_matrix %>%
  group_by(Word_Condition) %>%
  summarise(
    AD_Stat = ad.test(IR_Memorability)$statistic,
    p_value = ad.test(IR_Memorability)$p.value
  )
```

```
## # A tibble: 4 x 3
##   Word_Condition AD_Stat p_value
##   <fct>         <dbl>   <dbl>
## 1 HH           0.156 0.953
## 2 HL           0.875 0.0238
## 3 LH           1.55 0.000490
## 4 LL           0.413 0.331
```

```
aggregated_matrix %>%
  group_by(Sentence_Voice) %>%
  summarise(
    AD_Stat = ad.test(IR_Memorability)$statistic,
    p_value = ad.test(IR_Memorability)$p.value
  )
```

```
## # A tibble: 2 x 3
```

```
## Sentence_Voice AD_Stat p_value
## <fct>          <dbl>    <dbl>
## 1 Active        1.25 0.00280
## 2 Passive       1.13 0.00564
```

```
aggregated_matrix %>%
  group_by(Word_Condition) %>%
  summarise(
    AD_Stat = ad.test(WR_Accuracy)$statistic,
    p_value = ad.test(WR_Accuracy)$p.value
  )
```

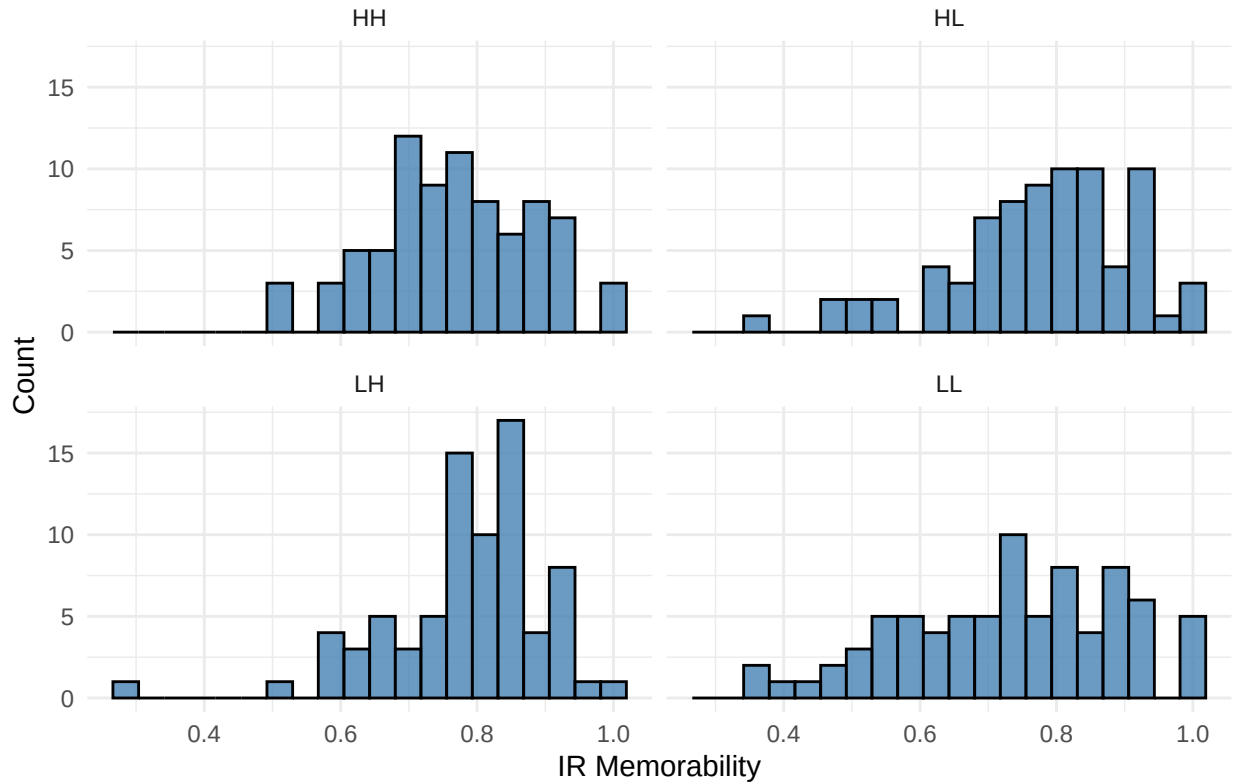
```
## # A tibble: 4 x 3
## Word_Condition AD_Stat p_value
## <fct>          <dbl>    <dbl>
## 1 HH           0.498 0.206
## 2 HL           0.520 0.181
## 3 LH           0.789 0.0389
## 4 LL           0.890 0.0219
```

```
aggregated_matrix %>%
  group_by(Sentence_Voice) %>%
  summarise(
    AD_Stat = ad.test(WR_Accuracy)$statistic,
    p_value = ad.test(WR_Accuracy)$p.value
  )
```

```
## # A tibble: 2 x 3
## Sentence_Voice AD_Stat p_value
## <fct>          <dbl>    <dbl>
## 1 Active        1.01 0.0113
## 2 Passive       1.26 0.00278
```

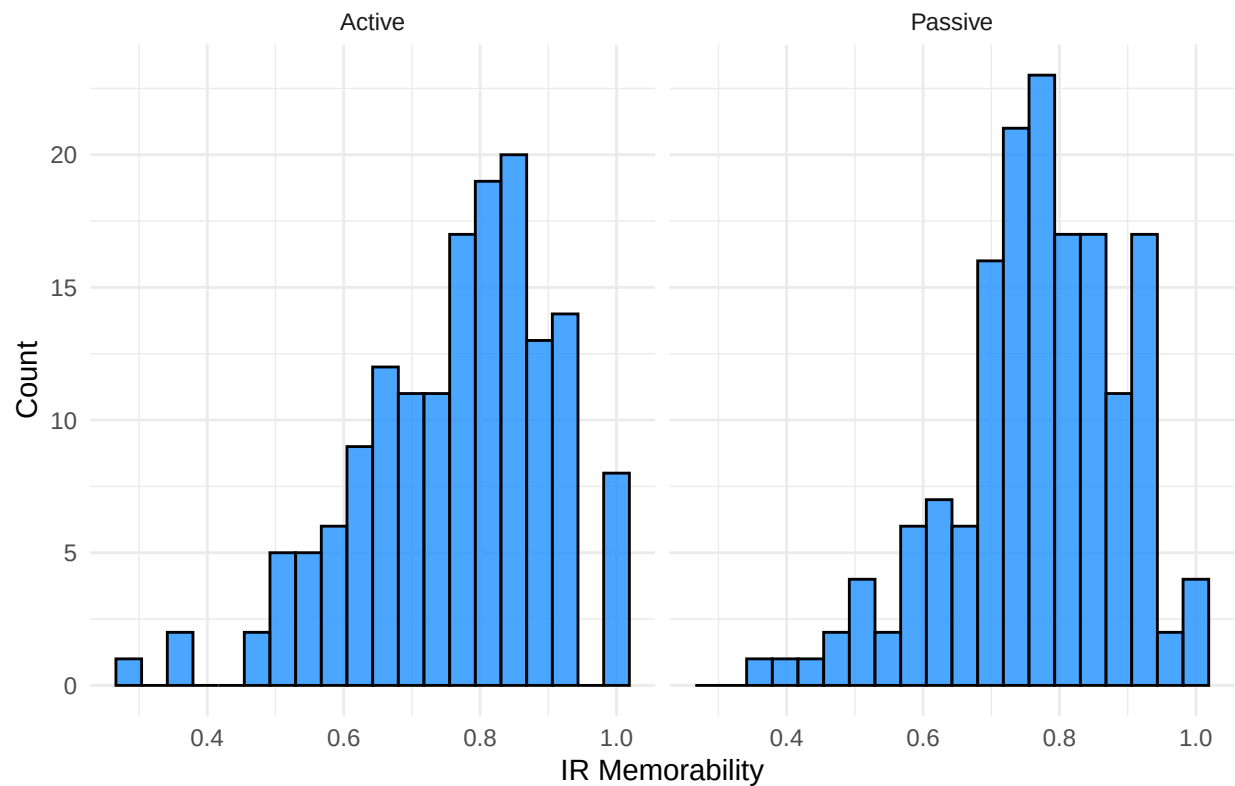
```
# Graph 1: Histogram of IR Memorability Scores
ggplot(aggregated_matrix, aes(x = IR_Memorability)) +
  geom_histogram(bins = 20, fill = "steelblue", color = "black", alpha = 0.8) +
  facet_wrap(~ Word_Condition, ncol = 2) +
  labs(
    title = "IR Memorability Distribution by Word Condition",
    x = "IR Memorability",
    y = "Count"
  ) +
  scale_y_continuous(limits = c(0, NA)) +
  theme_minimal()
```

IR Memorability Distribution by Word Condition



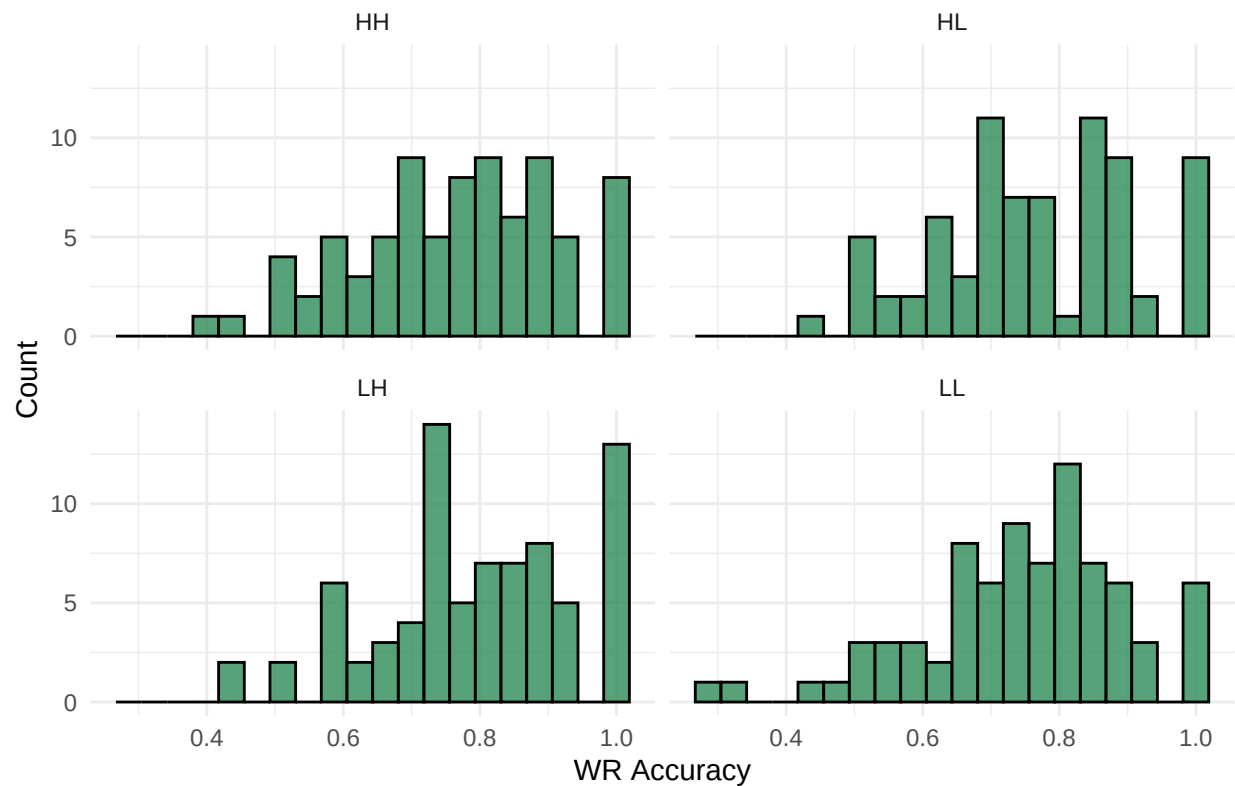
```
ggplot(aggregated_matrix, aes(x = IR_Memorability)) +
  geom_histogram(bins = 20, fill = "dodgerblue", color = "black", alpha = 0.8) +
  facet_wrap(~ Sentence_Voice, ncol = 2) +
  labs(
    title = "IR Memorability Distribution by Sentence Voice",
    x = "IR Memorability",
    y = "Count"
  ) +
  scale_y_continuous(limits = c(0, NA)) +
  theme_minimal()
```


IR Memorability Distribution by Sentence Voice



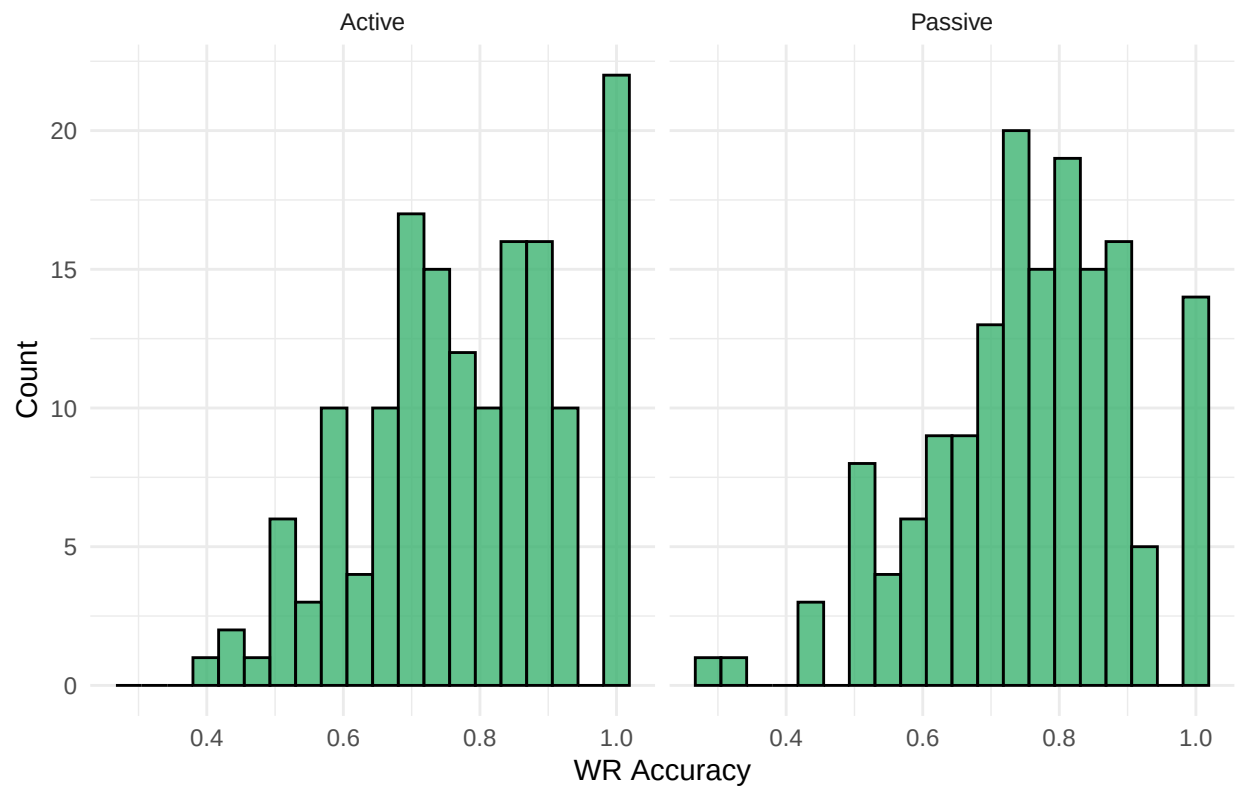
```
# Graph 2: Histogram of WR Accuracy Scores
ggplot(aggregated_matrix, aes(x = WR_Accuracy)) +
  geom_histogram(bins = 20, fill = "seagreen", color = "black", alpha = 0.8) +
  facet_wrap(~ Word_Condition, ncol = 2) +
  labs(
    title = "WR Accuracy Distribution by Word Condition",
    x = "WR Accuracy",
    y = "Count"
  ) +
  scale_y_continuous(limits = c(0, NA)) +
  theme_minimal()
```

WR Accuracy Distribution by Word Condition

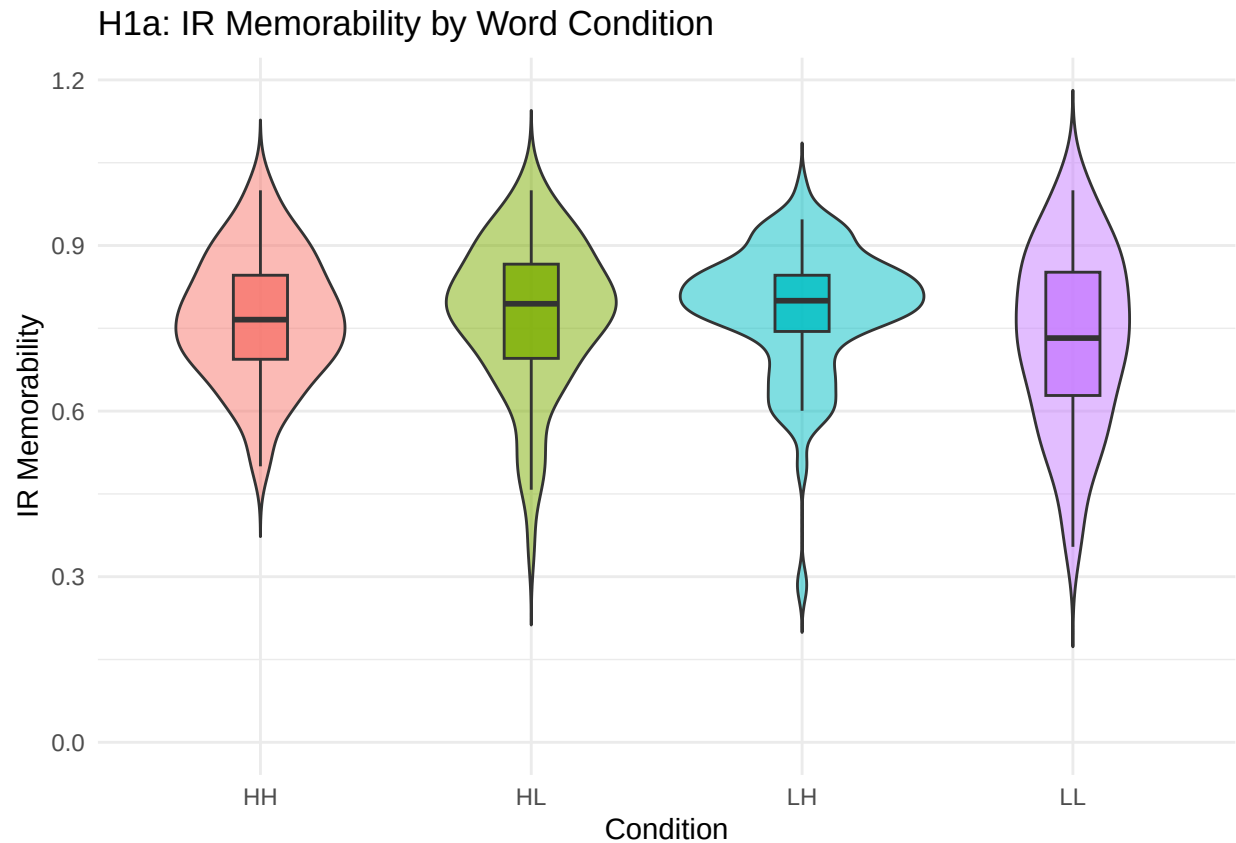


```
ggplot(aggregated_matrix, aes(x = WR_Accuracy)) +
  geom_histogram(bins = 20, fill = "mediumseagreen", color = "black", alpha = 0.8) +
  facet_wrap(~ Sentence_Voice, ncol = 2) +
  labs(
    title = "WR Accuracy Distribution by Sentence Voice",
    x = "WR Accuracy",
    y = "Count"
  ) +
  scale_y_continuous(limits = c(0, NA)) +
  theme_minimal()
```

WR Accuracy Distribution by Sentence Voice

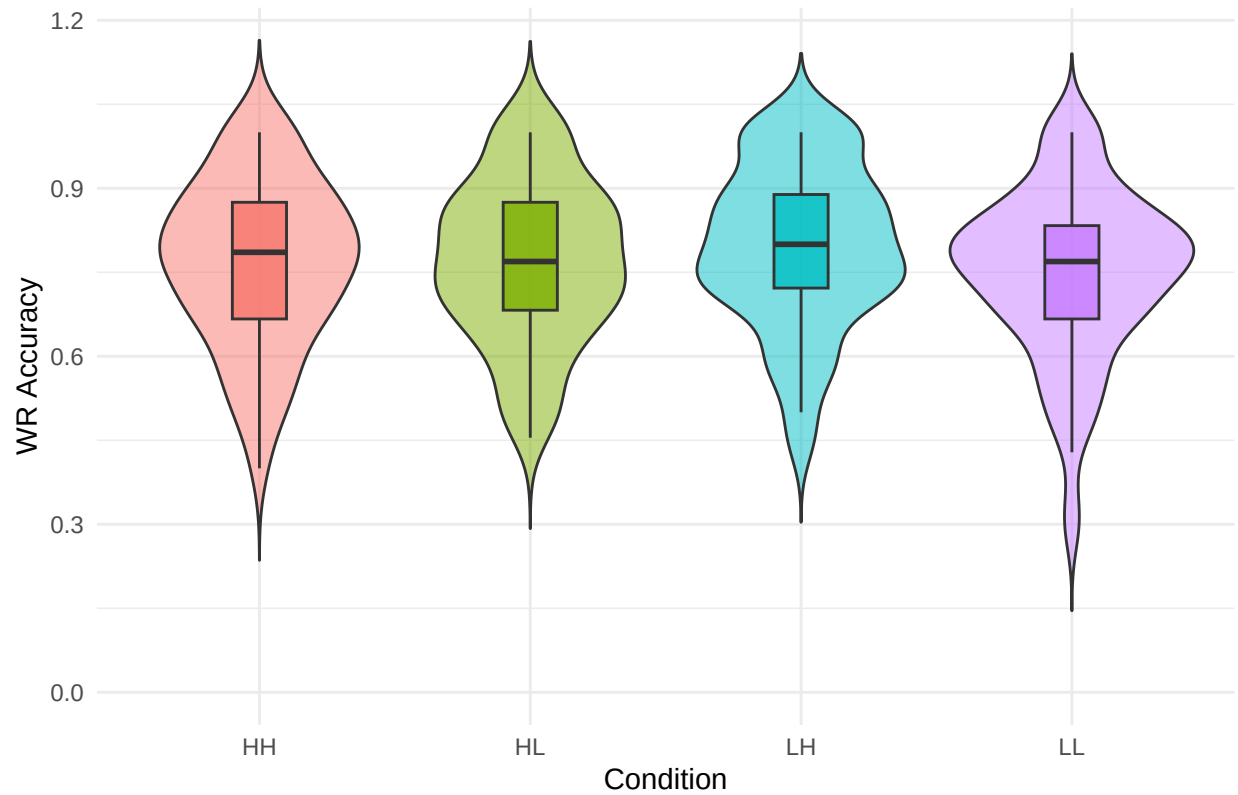


```
# Graph 3 (H1a): Violin & Boxplot for IR by Word Condition
ggplot(aggregated_matrix, aes(x = Word_Condition, y = IR_Memorability, fill = Word_Condition)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H1a: IR Memorability by Word Condition", x = "Condition", y = "IR Memorability") +
  theme_minimal() + theme(legend.position = "none")
```



```
# Graph 4 (H1b): Violin & Boxplot for WR by Word Condition
ggplot(aggregated_matrix, aes(x = Word_Condition, y = WR_Accuracy, fill = Word_Condition)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H1b: WR Accuracy by Word Condition", x = "Condition", y = "WR Accuracy") +
  theme_minimal() + theme(legend.position = "none")
```

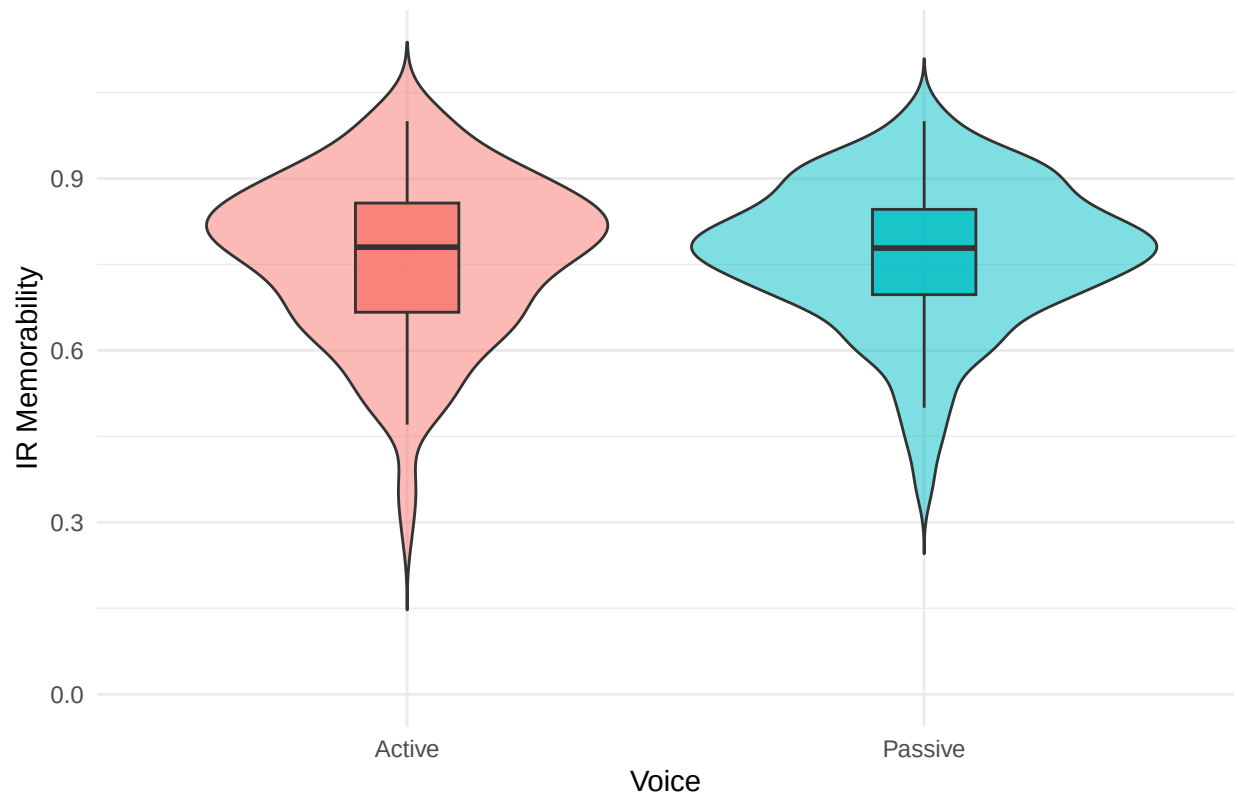
H1b: WR Accuracy by Word Condition



Graph 5 (H2a): Violin & Boxplot for IR by Voice

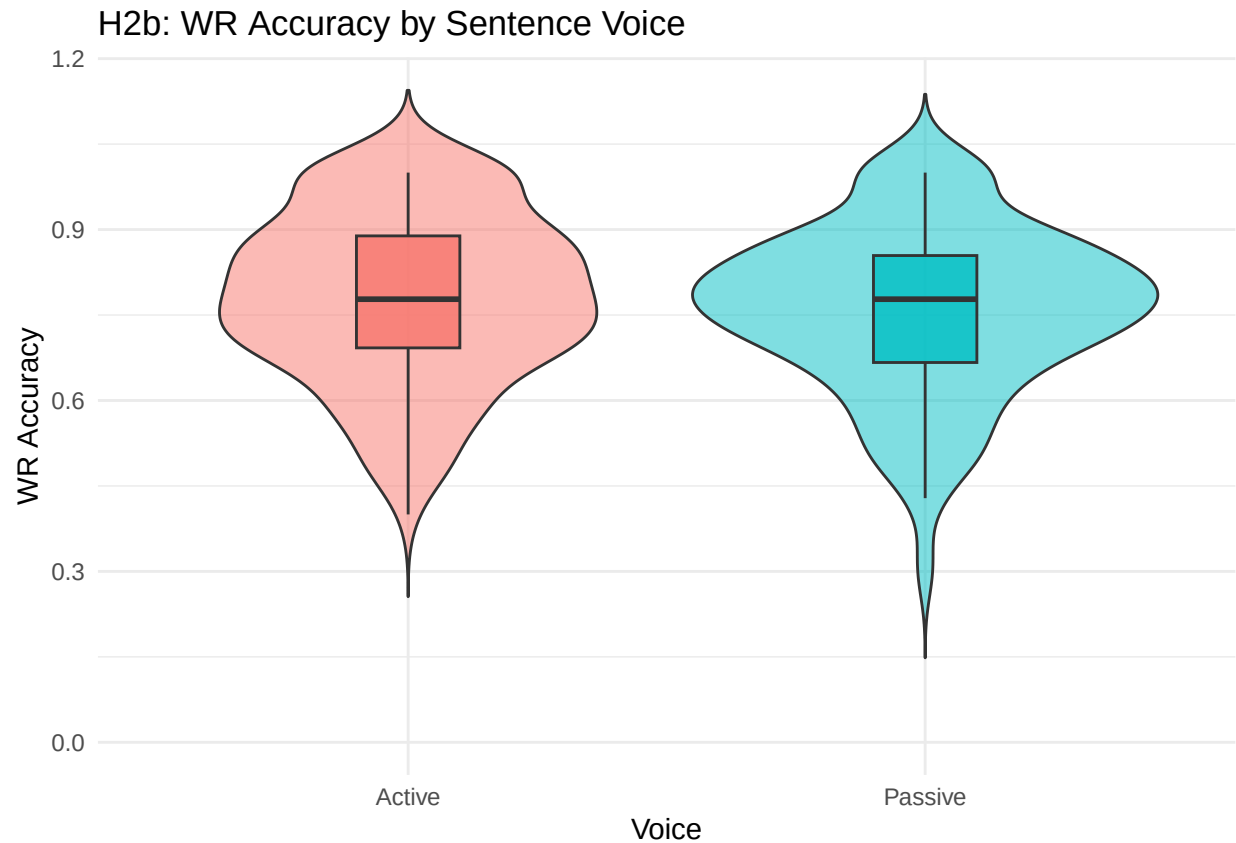
```
ggplot(aggregated_matrix %>% drop_na(Sentence_Voice), aes(x = Sentence_Voice, y = IR_Memorability, fill = Sentence_Voice)) +
  geom_violin(trim = FALSE, alpha = 0.5) +
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H2a: IR Memorability by Sentence Voice", x = "Voice", y = "IR Memorability") +
  theme_minimal() + theme(legend.position = "none")
```

H2a: IR Memorability by Sentence Voice



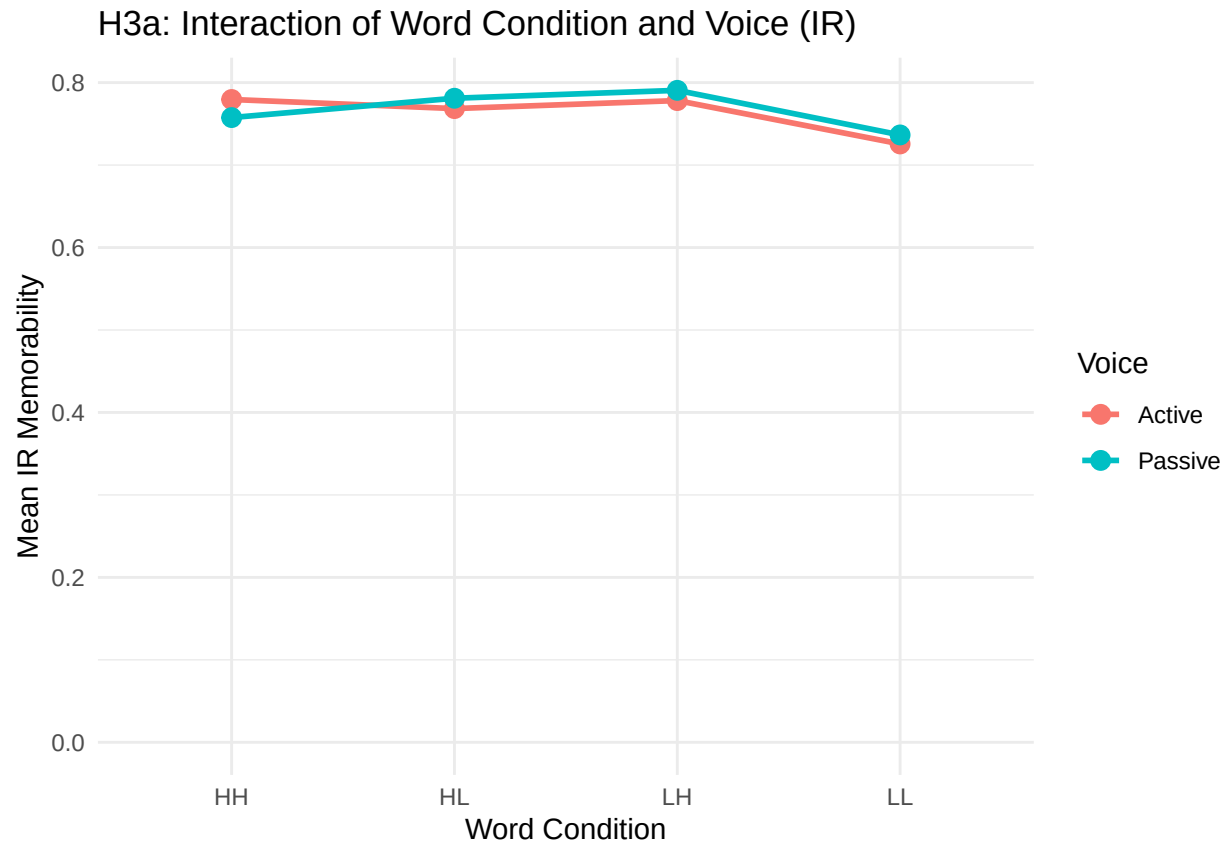
```
# Graph 6 (H2b): Violin & Boxplot for WR by Voice
```

```
ggplot(aggregated_matrix %>% drop_na(Sentence_Voice), aes(x = Sentence_Voice, y = WR_Accuracy, fill = S  
  geom_violin(trim = FALSE, alpha = 0.5) +  
  geom_boxplot(width = 0.2, outlier.shape = NA, alpha = 0.8) +  
  scale_y_continuous(limits = c(0, NA)) +  
  labs(title = "H2b: WR Accuracy by Sentence Voice", x = "Voice", y = "WR Accuracy") +  
  theme_minimal() + theme(legend.position = "none")
```



```
interaction_data <- aggregated_matrix %>%
  drop_na(Word_Condition, Sentence_Voice) %>%
  group_by(Word_Condition, Sentence_Voice) %>%
  summarise(
    Mean_IR = mean(IR_Memorability, na.rm = TRUE),
    Mean_WR = mean(WR_Accuracy, na.rm = TRUE),
    .groups = 'drop'
  )

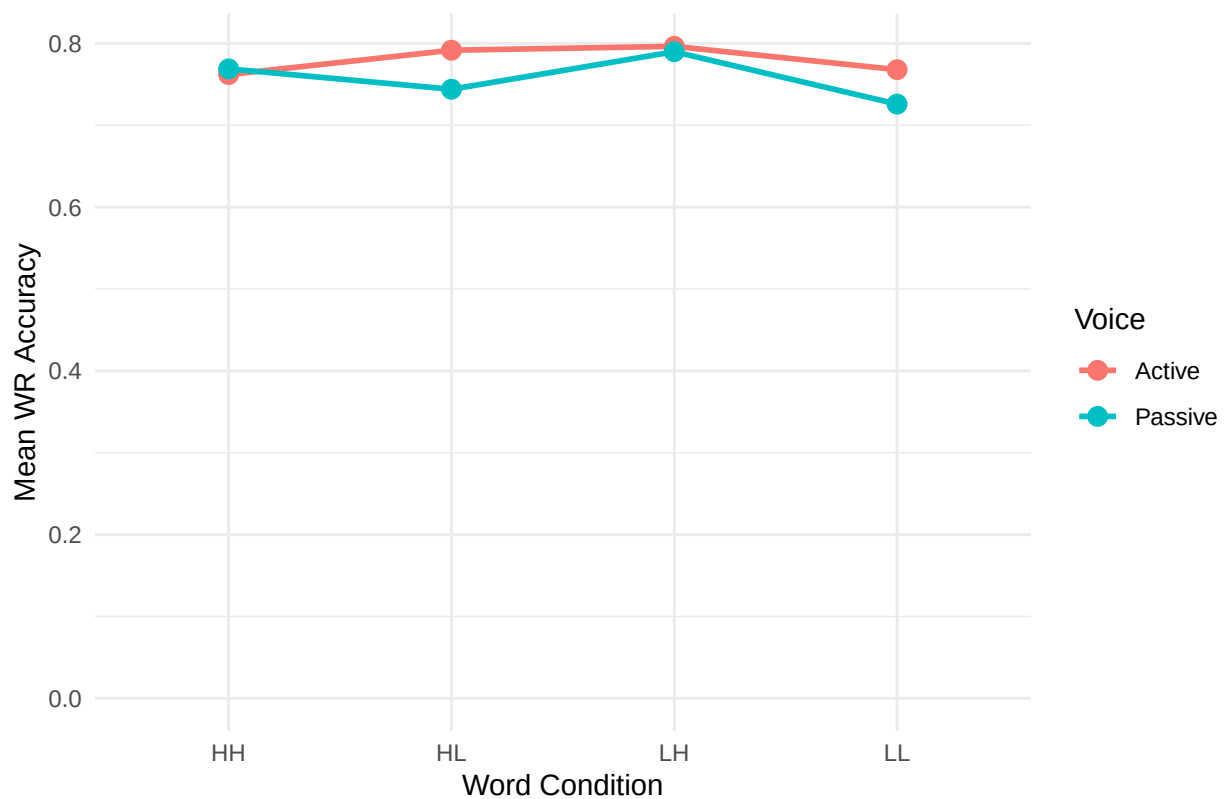
# Graph 7 (H3a): Interaction Plot for IR
ggplot(interaction_data, aes(x = Word_Condition, y = Mean_IR, color = Sentence_Voice, group = Sentence_Voice)) +
  geom_line(linewidth = 1) + geom_point(size = 3) +
  scale_y_continuous(limits = c(0, NA)) +
  labs(title = "H3a: Interaction of Word Condition and Voice (IR)", x = "Word Condition", y = "Mean IR") +
  theme_minimal()
```



Graph 8 (H3b): Interaction Plot for WR

```
ggplot(interaction_data, aes(x = Word_Condition, y = Mean_WR, color = Sentence_Voice, group = Sentence_Voice)) +  
  geom_line(linewidth = 1) + geom_point(size = 3) +  
  scale_y_continuous(limits = c(0, NA)) +  
  labs(title = "H3b: Interaction of Word Condition and Voice (WR)", x = "Word Condition", y = "Mean WR") +  
  theme_minimal()
```


H3b: Interaction of Word Condition and Voice (WR)



```
library(coin)
```

```
## Loading required package: survival
```

```
aggregated_scaled <- aggregated_matrix %>%
  mutate(
    IR_scaled = as.numeric(scale(IR_Memorability)),
    WR_scaled = as.numeric(scale(WR_Accuracy))
  )

# Calculate average scaled IR and WR scores and their standard errors for each word condition
effect_data <- aggregated_scaled %>%
  drop_na(Word_Condition, Sentence_Voice) %>%
  group_by(Word_Condition) %>%
  summarise(
    IR_mean = mean(IR_scaled, na.rm=TRUE),
    IR_se = sd(IR_scaled, na.rm=TRUE)/sqrt(n()),
    WR_mean = mean(WR_scaled, na.rm=TRUE),
    WR_se = sd(WR_scaled, na.rm=TRUE)/sqrt(n()),
    .groups="drop"
  )

# Graph 9 (H4): Word Memorability effects on IR vs WR
effect_data_long <- effect_data %>%
  pivot_longer(cols = c(IR_mean, WR_mean), names_to = "Metric", values_to = "Mean") %>%
```

```

mutate(SE = ifelse(Metric == "IR_mean", IR_se, WR_se))

long_data <- aggregated_scaled %>%
  drop_na(Word_Condition) %>%
  pivot_longer(
    cols = c(IR_scaled, WR_scaled),
    names_to = "Metric",
    values_to = "Score"
  ) %>%
  mutate(
    Metric = ifelse(Metric == "IR_scaled", "IR", "WR"),
    Word_Condition = as.factor(Word_Condition),
    Metric = as.factor(Metric),
    Score = as.numeric(Score)
  )

perm_test <- independence_test(
  Score ~ Word_Condition * Metric,
  data = long_data,
  distribution = approximate(nresample = 10000)
)

print("Permutation test to check if there is a difference in impact of word condition on the metrics:")

```

```
## [1] "Permutation test to check if there is a difference in impact of word condition on the metrics:"
```

```
perm_test
```

```

##
## Approximative General Independence Test
##
## data: Score by Word_Condition, Metric
## maxT = 2.886, p-value = 0.0195
## alternative hypothesis: two.sided

# checking if there's a bigger impact of word condition on IR than WR
effect_size <- function(data, response) {
  total_mean <- mean(data[[response]], na.rm = TRUE)
  ss_total <- sum((data[[response]] - total_mean)^2, na.rm = TRUE)
  ss_between <- data %>%
    group_by(Word_Condition) %>%
    summarise(mean_val = mean(.data[[response]], na.rm = TRUE),
              n = n(), .groups="drop") %>%
    summarise(sum(n * (mean_val - total_mean)^2)) %>% pull()
  return(ss_between / ss_total)
}

ir_eta <- effect_size(aggregated_scaled, "IR_scaled")
wr_eta <- effect_size(aggregated_scaled, "WR_scaled")

ir_eta

```

```
## [1] 0.02347823
```

```
wr_eta
```

```
## [1] 0.01271271
```

```
print("Difference in effect sizes - positive indicates higher effect on IR")
```

```
## [1] "Difference in effect sizes - positive indicates higher effect on IR"
```

```
ir_eta - wr_eta
```

```
## [1] 0.01076552
```

```
n_perm <- 10000
diffs <- numeric(n_perm)

for (i in 1:n_perm) {
  perm_data <- aggregated_scaled %>%
    mutate(Word_Condition = sample(Word_Condition))
  ir_perm <- effect_size(perm_data, "IR_scaled")
  wr_perm <- effect_size(perm_data, "WR_scaled")
  diffs[i] <- ir_perm - wr_perm
}

obs_diff <- ir_eta - wr_eta
p_value <- mean(abs(diffs) >= abs(obs_diff))
print("p-value to check if the increase is significant:")
```

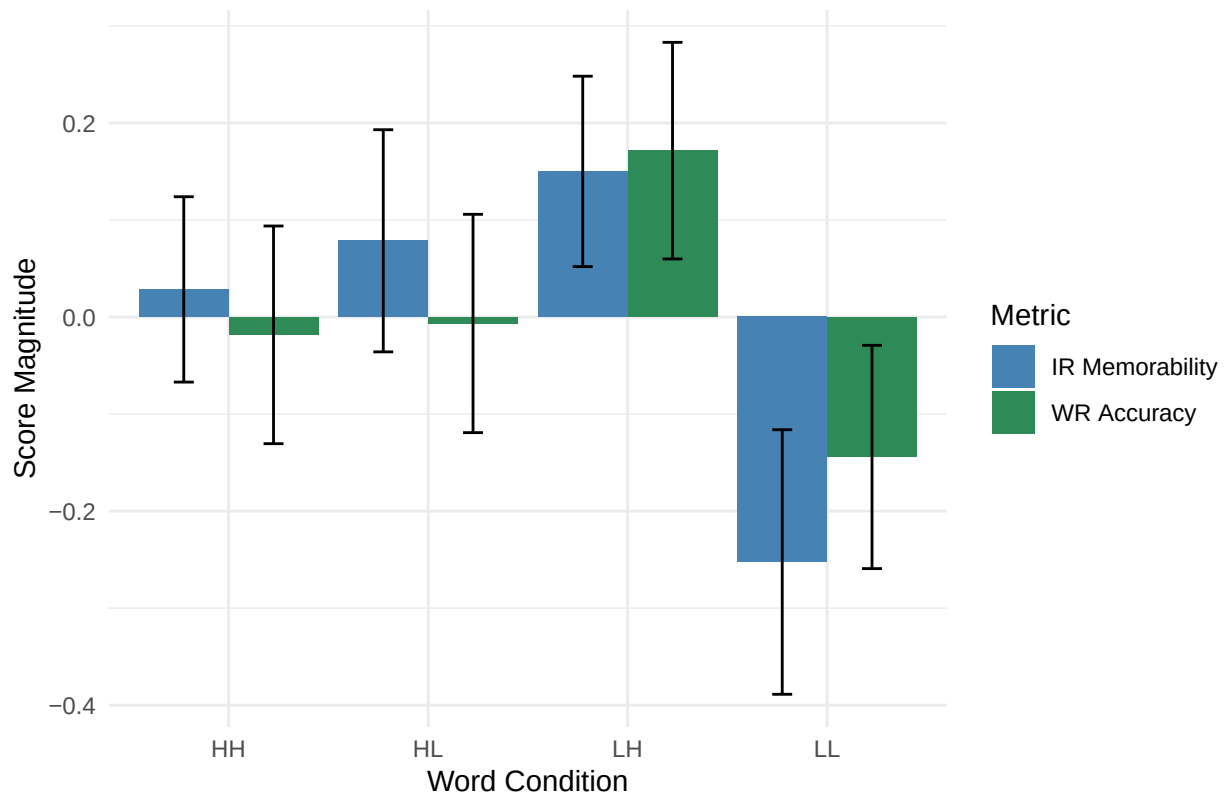
```
## [1] "p-value to check if the increase is significant:"
```

```
p_value
```

```
## [1] 0.2643
```

```
ggplot(effect_data_long, aes(x = Word_Condition, y = Mean, fill = Metric)) +
  geom_bar(stat = "identity", position = position_dodge()) +
  geom_errorbar(aes(ymin = Mean - SE, ymax = Mean + SE), width = 0.2, position = position_dodge(0.9)) +
  labs(title = "H4: Word Memorability Effect Magnitude (IR vs WR)", x = "Word Condition", y = "Score Magnitude") +
  scale_fill_manual(values = c("steelblue", "seagreen"), labels = c("IR Memorability", "WR Accuracy")) +
  theme_minimal()
```

H4: Word Memorability Effect Magnitude (IR vs WR)



Graph 10 (H5): Sentence Voice effects on IR vs WR

```
voice_effect <- aggregated_scaled %>%
```

```
drop_na(Sentence_Voice) %>%
```

```
group_by(Sentence_Voice) %>%
```

```
summarise(
```

```
  IR_mean = mean(IR_scaled, na.rm=TRUE),
```

```
  IR_se = sd(IR_scaled, na.rm=TRUE)/sqrt(n()),
```

```
  WR_mean = mean(WR_scaled, na.rm=TRUE),
```

```
  WR_se = sd(WR_scaled, na.rm=TRUE)/sqrt(n()),
```

```
  .groups="drop"
```

```
) %>%
```

```
pivot_longer(cols = c(IR_mean, WR_mean), names_to = "Metric", values_to = "Mean") %>%
```

```
mutate(SE = ifelse(Metric == "IR_mean", IR_se, WR_se))
```

```
ggplot(voice_effect, aes(x = Sentence_Voice, y = Mean, fill = Metric)) +
```

```
  geom_bar(stat = "identity", position = position_dodge()) +
```

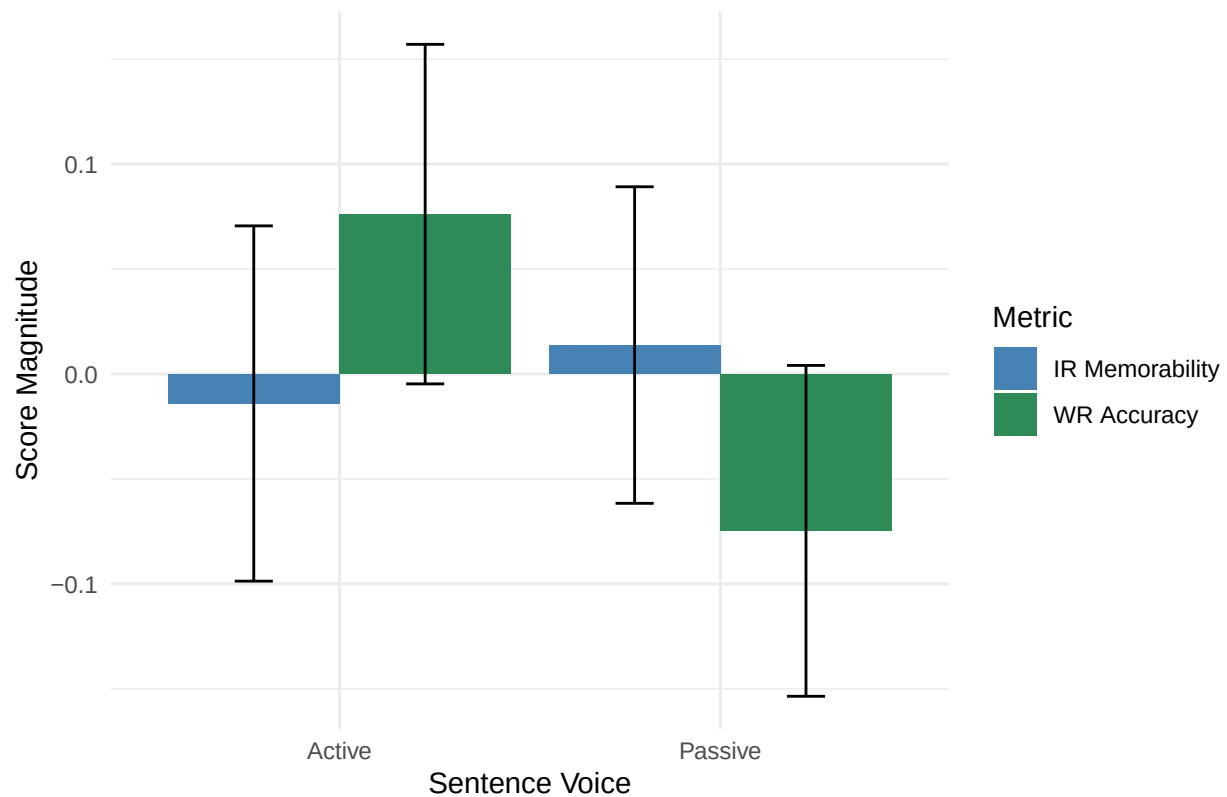
```
  geom_errorbar(aes(ymin = Mean - SE, ymax = Mean + SE), width = 0.2, position = position_dodge(0.9)) +
```

```
  labs(title = "H5: Sentence Voice Effect Magnitude (IR vs WR)", x = "Sentence Voice", y = "Score Magni")
```

```
  scale_fill_manual(values = c("steelblue", "seagreen"), labels = c("IR Memorability", "WR Accuracy")) +
```

```
  theme_minimal()
```

H5: Sentence Voice Effect Magnitude (IR vs WR)



```
long_voice <- aggregated_scaled %>%
  drop_na(Sentence_Voice) %>%
  pivot_longer(
    cols = c(IR_scaled, WR_scaled),
    names_to = "Metric",
    values_to = "Score"
  ) %>%
  mutate(
    Metric = ifelse(Metric == "IR_scaled", "IR", "WR"),
    Sentence_Voice = as.factor(Sentence_Voice),
    Metric = as.factor(Metric),
    Score = as.numeric(Score)
  )

voice_perm_test <- independence_test(
  Score ~ Sentence_Voice * Metric,
  data = long_voice,
  distribution = approximate(nresample = 10000)
)

print("Permutation test to check if there is a difference in impact of sentence voice on the metrics:")
```

```
## [1] "Permutation test to check if there is a difference in impact of sentence voice on the metrics:"
```

```
voice_perm_test
```

```
##  
## Approximative General Independence Test  
##  
## data: Score by Sentence_Voice, Metric  
## maxT = 0.7702, p-value = 0.6903  
## alternative hypothesis: two.sided
```

```
# Descriptive stats by Word Condition  
desc_word <- aggregated_matrix %>%  
  drop_na(Word_Condition) %>%  
  group_by(Word_Condition) %>%  
  summarise(  
    IR_mean = mean(IR_Memorability, na.rm = TRUE),  
    IR_sd = sd(IR_Memorability, na.rm = TRUE),  
    IR_median = median(IR_Memorability, na.rm = TRUE),  
  
    WR_mean = mean(WR_Accuracy, na.rm = TRUE),  
    WR_sd = sd(WR_Accuracy, na.rm = TRUE),  
    WR_median = median(WR_Accuracy, na.rm = TRUE),  
  
    n = n(),  
    .groups = "drop"  
  )  
  
print(desc_word)
```

```
## # A tibble: 4 x 8  
##   Word_Condition IR_mean IR_sd IR_median WR_mean WR_sd WR_median     n  
##   <fct>          <dbl> <dbl>    <dbl>  <dbl> <dbl>    <dbl> <int>  
## 1 HH           0.768 0.113    0.766   0.765 0.146    0.786    80  
## 2 HL           0.775 0.132    0.794   0.767 0.143    0.769    76  
## 3 LH           0.784 0.115    0.8     0.793 0.144    0.8      78  
## 4 LL           0.731 0.161    0.732   0.747 0.149    0.769    79
```

```
# Descriptive stats by Sentence Voice  
desc_voice <- aggregated_matrix %>%  
  drop_na(Sentence_Voice) %>%  
  group_by(Sentence_Voice) %>%  
  summarise(  
    IR_mean = mean(IR_Memorability, na.rm = TRUE),  
    IR_sd = sd(IR_Memorability, na.rm = TRUE),  
    IR_median = median(IR_Memorability, na.rm = TRUE),  
  
    WR_mean = mean(WR_Accuracy, na.rm = TRUE),  
    WR_sd = sd(WR_Accuracy, na.rm = TRUE),  
    WR_median = median(WR_Accuracy, na.rm = TRUE),  
  
    n = n(),  
    .groups = "drop"  
  )
```

```
print(desc_voice)
```

```
## # A tibble: 2 x 8
##   Sentence_Voice IR_mean IR_sd IR_median WR_mean WR_sd WR_median     n
##   <fct>          <dbl> <dbl>     <dbl>   <dbl> <dbl>   <dbl> <int>
## 1 Active          0.763 0.140     0.780   0.779 0.147   0.778   155
## 2 Passive          0.766 0.126     0.779   0.757 0.144   0.778   158
```

```
kruskal_ir_word <- kruskal.test(IR_Memorability ~ Word_Condition, data = aggregated_matrix)
print(kruskal_ir_word)
```

```
##
##   Kruskal-Wallis rank sum test
##
## data:  IR_Memorability by Word_Condition
## Kruskal-Wallis chi-squared = 6.2173, df = 3, p-value = 0.1015
```

```
kruskal_wr_word <- kruskal.test(WR_Accuracy ~ Word_Condition, data = aggregated_matrix)
print(kruskal_wr_word)
```

```
##
##   Kruskal-Wallis rank sum test
##
## data:  WR_Accuracy by Word_Condition
## Kruskal-Wallis chi-squared = 3.4748, df = 3, p-value = 0.3241
```

```
mw_ir_voice <- wilcox.test(
  IR_Memorability ~ Sentence_Voice,
  data = aggregated_matrix,
  exact = FALSE
)

print(mw_ir_voice)
```

```
##
##   Wilcoxon rank sum test with continuity correction
##
## data:  IR_Memorability by Sentence_Voice
## W = 12256, p-value = 0.9895
## alternative hypothesis: true location shift is not equal to 0
```

```
mw_wr_voice <- wilcox.test(
  WR_Accuracy ~ Sentence_Voice,
  data = aggregated_matrix,
  exact = FALSE
)

print(mw_wr_voice)
```

```

##
## Wilcoxon rank sum test with continuity correction
##
## data: WR_Accuracy by Sentence_Voice
## W = 13159, p-value = 0.2531
## alternative hypothesis: true location shift is not equal to 0

# Create categorical correctness variables
processed_trials <- raw_dataset %>%
  filter(isTarget == TRUE) %>%
  arrange(participant_ID, Stimulus, Timestamp) %>%
  group_by(participant_ID, Stimulus) %>%
  mutate(
    Word_Condition = case_when(
      str_detect(Stimulus, "^HH_") ~ "HH",
      str_detect(Stimulus, "^HVL_") ~ "HL",
      str_detect(Stimulus, "^LVH_") ~ "LH",
      str_detect(Stimulus, "^LVL_") ~ "LL"
    ),
    Word_Condition = factor(Word_Condition, levels = c("HH", "HL", "LH", "LL")),

    Sentence_Voice = case_when(
      str_detect(Stimulus, "_A$") ~ "Active",
      str_detect(Stimulus, "_P$") ~ "Passive",
      TRUE ~ NA_character_
    ),
    Sentence_Voice = factor(Sentence_Voice, levels = c("Active", "Passive")),

    Accuracy.IR = suppressWarnings(as.numeric(Accuracy.IR)),
    Accuracy.WR = suppressWarnings(as.numeric(Accuracy.WR)),
    prev_IR = lag(Accuracy.IR)
  ) %>%
  ungroup()

ir_data <- processed_trials %>%
  filter(Event == "IR pressed") %>%
  mutate(
    IR_Result = case_when(
      isRepeat == TRUE & Accuracy.IR == 1 ~ "Correct",
      is.na(isRepeat) ~ "Incorrect",
      TRUE ~ NA_character_
    )
  ) %>%
  filter(!is.na(IR_Result))

wr_data <- processed_trials %>%
  filter(Event == "WR pressed", prev_IR == 1) %>%
  mutate(
    WR_Result = case_when(
      Accuracy.WR == 1 ~ "Correct",
      Accuracy.WR == 0 ~ "Incorrect",
      TRUE ~ NA_character_
    )
  ) %>%

```



```

filter(!is.na(WR_Result))

ir_word_table <- table(ir_data$Word_Condition, ir_data$IR_Result)
fisher_ir_word <- fisher.test(ir_word_table)
print(fisher_ir_word)

##
## Fisher's Exact Test for Count Data
##
## data: ir_word_table
## p-value = 0.001891
## alternative hypothesis: two.sided

wr_word_table <- table(wr_data$Word_Condition, wr_data$WR_Result)
fisher_wr_word <- fisher.test(wr_word_table)
print(fisher_wr_word)

##
## Fisher's Exact Test for Count Data
##
## data: wr_word_table
## p-value = 0.2951
## alternative hypothesis: two.sided

ir_voice_table <- table(ir_data$Sentence_Voice, ir_data$IR_Result)
fisher_ir_voice <- fisher.test(ir_voice_table)
print(fisher_ir_voice)

##
## Fisher's Exact Test for Count Data
##
## data: ir_voice_table
## p-value = 0.4801
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
## 0.6474632 1.2316031
## sample estimates:
## odds ratio
## 0.8936385

wr_voice_table <- table(wr_data$Sentence_Voice, wr_data$WR_Result)
fisher_wr_voice <- fisher.test(wr_voice_table)
print(fisher_wr_voice)

##
## Fisher's Exact Test for Count Data
##
## data: wr_voice_table
## p-value = 0.1859

```

```
## alternative hypothesis: true odds ratio is not equal to 1
## 95 percent confidence interval:
## 0.9516112 1.2989532
## sample estimates:
## odds ratio
## 1.111716
```

```
# Create categorical correctness variables
```

```
ir_word_table <- table(ir_data$Word_Condition, ir_data$IR_Result)
chisq_ir_word <- chisq.test(ir_word_table)
print(chisq_ir_word)
```

```
##
## Pearson's Chi-squared test
##
## data: ir_word_table
## X-squared = 15.516, df = 3, p-value = 0.001424
```

```
wr_word_table <- table(wr_data$Word_Condition, wr_data$WR_Result)
chisq_wr_word <- chisq.test(wr_word_table)
print(chisq_wr_word)
```

```
##
## Pearson's Chi-squared test
##
## data: wr_word_table
## X-squared = 3.669, df = 3, p-value = 0.2995
```

```
ir_voice_table <- table(ir_data$Sentence_Voice, ir_data$IR_Result)
chisq_ir_voice <- chisq.test(ir_voice_table)
print(chisq_ir_voice)
```

```
##
## Pearson's Chi-squared test with Yates' continuity correction
##
## data: ir_voice_table
## X-squared = 0.4034, df = 1, p-value = 0.5253
```

```
wr_voice_table <- table(wr_data$Sentence_Voice, wr_data$WR_Result)
chisq_wr_voice <- chisq.test(wr_voice_table)
print(chisq_wr_voice)
```

```
##
## Pearson's Chi-squared test with Yates' continuity correction
##
## data: wr_voice_table
## X-squared = 1.7482, df = 1, p-value = 0.1861
```

```

ir_wr_counts <- raw_dataset %>%
  arrange(participant_ID, Stimulus, Timestamp) %>%
  group_by(participant_ID) %>%
  mutate(
    Accuracy.IR = suppressWarnings(as.numeric(Accuracy.IR)),
    Accuracy.WR = suppressWarnings(as.numeric(Accuracy.WR)),
    prev_IR = lag(Accuracy.IR)
  ) %>%
  ungroup() %>%
  summarise(
    IR_Total = sum(Event == "IR pressed", na.rm = TRUE),
    IR_Correct = sum(Event == "IR pressed" & Accuracy.IR == 1 & isRepeat==TRUE, na.rm = TRUE),
    IR_Incorrect = sum(Event == "IR pressed" & Accuracy.IR == 0, na.rm = TRUE),

    WR_Total = sum(Event == "WR pressed" & prev_IR == 1, na.rm = TRUE),
    WR_Correct = sum(Event == "WR pressed" & Accuracy.WR == 1 & prev_IR == 1, na.rm = TRUE),
    WR_Incorrect = sum(Event == "WR pressed" & Accuracy.WR == 0 & prev_IR == 1, na.rm = TRUE)
  )

print(ir_wr_counts)

```

```

## # A tibble: 1 x 6
##   IR_Total IR_Correct IR_Incorrect WR_Total WR_Correct WR_Incorrect
##   <int>     <int>       <int>    <int>     <int>       <int>
## 1     4342     4174         168     3733     2875        858

```

Contributions:

Bhavani: Data processing and formulae Nishita: Plotting Graphs and Debugging Tejasvini: Organizing the graphs and plotting graphs